
memset(result,0,100);
for_p2p_s.get_bindkey = true;
link_audio_file(bind_key);
strcpy(for_p2p_s.yi_cloud_info.bindkey,bind_key);
region_id_00 = judge_bindkey(for_p2p_s.yi_cloud_info.bindkey);
choose_server(region_id_00);
pcVar1 = sysTime();
_Var2 = getpid();
udp://127.0.0.1:13377
L-CS09A/SV822D-V1
Mii Need
Some 0-Days
KuWifi
struct sys_wifi_config
Length: 164 (0xa4) Alignment: 4
char[32] ssid
char[32] mode
char[32] passwd
wifi_enc_type enc_type
char[32] ap_ssid
char[32] ap_pswd
} pack()
http客户端
PN26QQ1AWSIUG
1844HT
HER643
CP1114948
SV822D
udp://127.0.0.1:13377
2024-04-05 14:24:36
PROJECT MAG

Contents

YOU WOULDN'T GAMBLE ON A ROUTER

Or would you? It is at the focal point of the LAN, making it the perfect candidate for a LAN-gambling-party Blackjack host.
Note: *Gamble responsibly!*

Target Router

This project started as a general RE/VR project, focusing on bugs over the LAN. Aliexpress always has cheap embedded goodies with questionable security. Sorting the search results for '*router*' by '*lowest price first*', I ended up with this masterpiece for £11 (flames not included).

Under the hood, it has a Mediatek MT7628KN chip (MIPS architecture), a labelled UART, and an SPI EEPROM memory chip. The command line on the UART allowed extraction of the firmware from the EEPROM. Then, *basefind2* was used to work out the base address, and it was loaded into *Ghidra* (no symbols). Its underlying OS is *eCos*, and it's built around an old Realtek SDK.

Finding CVE-2012-5959

By auditing *strcpy()* calls, I spotted a simple stack-based buffer overflow in the SSDP parser in the UPnP library. It turns out the ancient SDK uses a version of *libupnp* vulnerable to an old CVE! To trigger a crash, send:

```
M-SEARCH * HTTP/1.1
HOST:239.255.255.250:1900
MAN:"ssdp:discover"
MX:3
ST:uuid:AAAAAAAA...AAAAAAAAA
```

As the router has no mitigations (like the good ol' days), this bug enables unauthenticated RCE over the LAN.

Brainstorming Exploit

At the moment, we overwrite the return address with garbage that is not mapped into memory, we need to jump somewhere useful. We could put a shellcode in our *uuid* buffer and jump to that - but it's definitely not big enough for a Blackjack server.

Alternatively, we can use ROP (Return-Oriented-Programming) gadget(s) to construct a larger shellcode in memory via multiple requests. Using the following gadget, we can write four bytes wherever we please (provided it is mapped):

```
0x8013be14:
  sw $s0, ($s1); lw $ra, 0xc($sp);
  move $v0, $s0; lw $s2, 8($sp);
  lw $s1, 4($sp); lw $s0, ($sp);
  jr $ra; addiu $sp, $sp, 0x10;
```

The gadget increments the stack pointer by 0x10 bytes. To avoid a crash, we set the return address after the gadget runs to the address of the stack frame two frames up (each frame is 8 bytes). So, we can now build our larger shellcode in memory via multiple requests.

XOR Decoder

MIPS binaries tend to contain lots of zeros. Because of the string-based nature of our overflow, zeros are a no-go as they terminate the string early. Our write ROP gadget simply stores the contents of $s0 at the address in $s1, both taken directly from the buffer we send - so the payload can't contain zeros.

Our payload should be small, placing a simple XOR decoder at the start should suffice. This runs before the payload, iterating over the rest of the payload and XOR'ing. We then wait to let the caches naturally flush (thanks MIPS), and jump to the decoded payload.

Shellcode Location

In *eCos*, each thread gets its own memory area for its stack. The size of this region for the *HTTP_proc* thread is 16384 bytes, and during normal operation this hovers around 40% utilisation. We can borrow a chunk of this stack region for our Blackjack payload.

Building Blackjack

To get our server running, we have two options: create a new thread or hijack an existing one. The thread named *cpuload*, which can only be activated via the shell, is perfect for this. We can hijack this thread by invoking the *create_thread* function with identical addresses, substituting our blackjack server as the thread function. As the *cpuload* thread is always suspended, the router carries on as normal.

For the socket functionality, we must reverse engineer the firmware and extract the address of anything we need (*listen(), recv(), bind()*, etc). These addresses can then be hard-coded in our payload, and used in our C code as functions. Most of the server goes together like a standard UDP server example, plus some extra logic for the game and player management.

Compiling BlackJack

Our C program must run on MIPS, despite being compiled on an x86 processor, so we'll need to cross-compile. For this, I used *crosstools-ng* to construct a toolchain, allowing me to compile C on x86 into a MIPS binary. The payload location is set in the linker file we give to the compiler, so the code knows where it is. *Note: There are also pre-built eCos toolchains available at https://ecos.sourceware.org/build-toolchain.html.*

LAN-Gambling

With our payload put together, it ends up being around 6000 bytes, so it fits perfectly into the empty stack space mentioned earlier. Now, we can send multiple requests to construct the compiled Blackjack server into memory. Once complete, we can jump to this memory, decode the payload, wait for caches to flush, then jump to the decoded payload which hijacks the *cpuload* thread and runs the server function.

Once the thread is running, players can connect to the server with *netcat* - once all players are in, the game can begin!

```
    ___  __           __       _         __
   / _ )/ /__ _____  / /__    (_)__ ____/ /__
  / _  / / _ `/ __/ /  '_/   / / _ `/ __/  '_/
 /____/_/\_,_/\__/ /_/\_\ __/ /\_,_/\__/_/\_\
                         |___/
Welcome player 1!
Enter player count (including yourself)
> 3

Waiting for 2 other players to join...
Game starting!

- Player 1 has $250
- Player 2 has $250
- Player 3 has $250

Place your bet player 1...
> 10
Player 1 has bet $10

Player 2 is placing their bet
Player 2 has bet $10

Player 3 is placing their bet
Player 3 has bet $10

Player 1 cards:
 _______   _______
|Q _ _  | |J  ^   |
| ( v ) | |  / \  |
|  \ /  | |  \ /  |
|   .   | |   .   |
|______Q| |______J|

Player 2 cards:
 _______   _______
|7  ^   | |3 _ _  |
|  / \  | | ( v ) |
|  \ /  | |  \ /  |
|   .   | |   .   |
|______7| |______3|

Player 3 cards:
 _______   _______
|10 ^   | |A _ _  |
|  / \  | | ( v ) |
|  \ /  | |  \ /  |
|   .   | |   .   |
|_____10| |______A|

Player 3 has blackjack!

Dealers card:
 _______
|K  .   |
|  /.\  |
| (_._) |
|   |   |
|______K|

Player 1's move, current hand:
 _______   _______
|Q _ _  | |J  ^   |
| ( v ) | |  / \  |
|  \ /  | |  \ /  |
|   .   | |   .   |
|______Q| |______J|

Stick or twist?
> s

Player 1 chose to stick
Player 1 final score: 20

Player 2's move, current hand:
 _______   _______
|7  ^   | |3 _ _  |
|  / \  | | ( v ) |
|  \ /  | |  \ /  |
|   .   | |   .   |
|______7| |______3|

Player 2 chose to twist, current hand:
 _______   _______   _______
|7  ^   | |3 _ _  | |8  .   |
|  / \  | | ( v ) | |  /.\  |
|  \ /  | |  \ /  | | (_._) |
|   .   | |   .   | |   |   |
|______7| |______3| |______8|

Player 2 chose to stick
Player 2 final score: 18

All players done...

The dealer's full hand is:
 _______   _______
|K  .   | |3  .   |
|  /.\  | |  /.\  |
| (_._) | | (_._) |
|   |   | |   |   |
|______K| |______3|

Dealer is twisting

Dealer's current hand:
 _______   _______   _______
|K  .   | |3  .   | |A  .   |
|  /.\  | |  /.\  | |  /.\  |
| (_._) | | (_._) | | (_._) |
|   |   | |   |   | |   |   |
|______K| |______3| |______A|

Dealer is twisting

Dealer's current hand:
 _______   _______   _______   _______
|K  .   | |3  .   | |A  .   | |J _ _  |
|  /.\  | |  /.\  | |  /.\  | | ( v ) |
| (_._) | | (_._) | | (_._) | |  \ /  |
|   |   | |   |   | |   |   | |   .   |
|______K| |______3| |______A| |______J|

Dealer is bust!

You won $10!
```

IF IT HAS A STREAM, IT CAN PLAY DOOM

At this point, it is a pretty well-proven fact that "*if it has a screen, it can play doom*". In this article, I extend the scope of devices covered by this rule by remotely hacking an IoT camera to stream DOOM - without touching the firmware!

Target

This investigation focuses on generic cameras using the Yi IoT app, usually found on eBay, Amazon, and AliExpress. (I bought mine from AliExpress for £15.) Identical cameras using different apps are beyond this article's scope.

These cameras, based on an Anyka SoC (with an ARM MCU) running Linux, come in various forms. A root shell is easily obtained via UART test points.

They are packed with interesting features, such as cloud control, two-way audio, motion tracking, etc.

There are two modes: *Hotspot* and *Cloud*. In Hotspot mode, you connect to an access point hosted by the camera to interact with and view the stream. Cloud mode allows remote access via relay servers. This article assumes the camera is in Hotspot mode, offering a larger attack surface. Most of the functionality is performed by the *anyka_ipc* binary, and several imported shared objects.

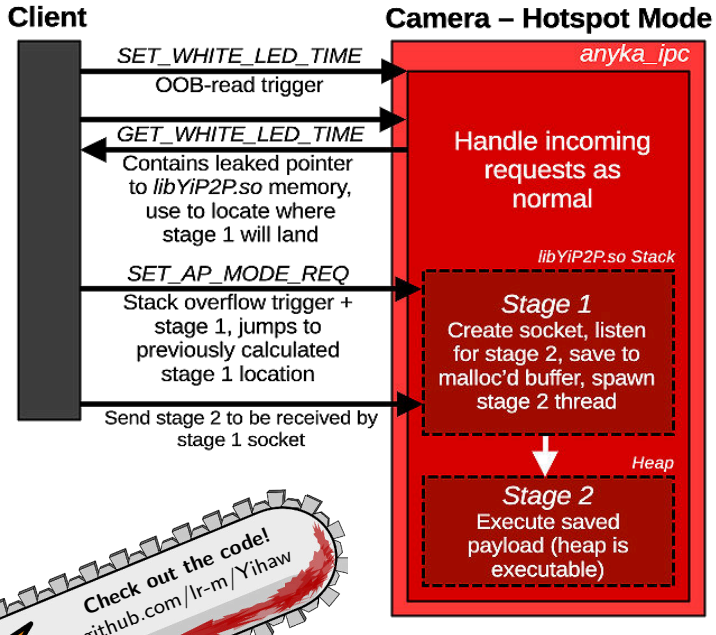
The main *anyka_ipc* binary has NX on the stack and ASLR on only the stack/shared objects. The heap is already executable, which is handy. As ASLR is only present on the shared libraries, we only need to account for their varying memory locations - a leak would be nice if we find memory corruption.

Bugs

1. *Stack Overflow*: A message handler in the camera has a trivial `strcpy()` stack overflow. The overflow occurs in an executable shared library memory region (`libYiP2P.so`) - as string-based, no null bytes in payload.
2. *File Write*: The software update handler accepts files through port 6000. An MD5 hash is provided and subsequently checked against that of the received file. If the provided MD5 hash check fails, the file isn't deleted.
3. *OOB-Read*: A missing bounds check in the message header offset lets us read beyond the buffer containing the incoming message. This lets us leak useful addresses remotely (to work out the location of the packet in executable memory!).

Getting Arbitrary Code Execution

Lets use the primitives we have above to get ACE. Here are the exploit steps:



Check out the code!
github.com/lr-m/Yihaw

Hijacking the Stream

Now that we can execute arbitrary code, we can overwrite the *yuv420p* frames from the sensor. There are three main threads for handling and sending footage to the app:

1. `capture_thread`: Gets *yuv420p* data from the sensor and adds it to the encode list.
2. `encode_thread`: Takes *yuv420p* data from the encode list and sends it to the hardware encoder to convert it into an *h264* frame.
3. `yi_live_video_thread`: Sends the *h264* encoded frame to the app.

Stage 2 exits those threads by setting global flags that cause them to exit, then starts our own versions. The difference is that our `capture_thread` gets *yuv420p* data from a non-sensor source, allowing us to control the data fed to the hardware encoder (which converts *yuv420p* data into *h264*).

IPC

To enable communication between our stage 2 running in *anyka_ipc* and the DOOM binary, I used two regions of file-backed shared memory: one for *yuv420p* frames from DOOM to stage 2, and one for controls from stage 2 to DOOM. I mapped the memory into both processes using `mmap()`.

Porting and Compiling DOOM

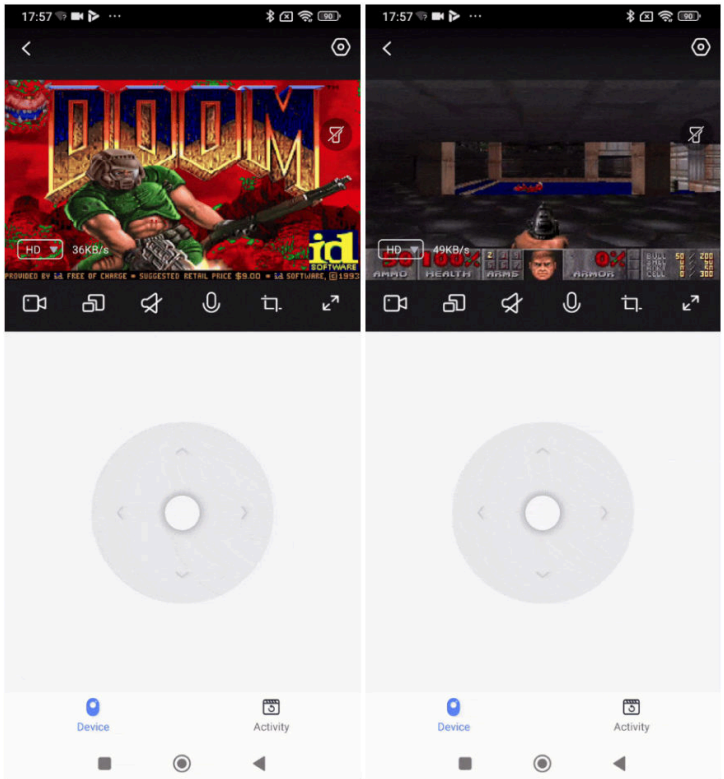
This was easier than expected thanks to *doomgeneric*, which simplifies porting DOOM by requiring the implementation of a few functions. I omitted `DG_SleepMs` and `DG_GetTicksMs` as they are straightforward:

- `DG_Init` (*Init your platform*): Precompute lookup tables for RGB (from DOOM) to *yuv420p* (for the app) and `mmap()` shared memory for the framebuffer and controls.
- `DG_DrawFrame` (*Copy framebuffer to platform screen*): Convert the RGB framebuffer to *yuv420p* using lookup tables and write to framebuffer shared memory.
- `DG_GetKey` (*Provide keyboard events*): Get current app buttons pressed from shared memory sent from our stage 2.

With our functions implemented, we simply cross-compile DOOM using:

```
arm-linux-gnueabi-gcc -static -Ofast *.c -o doom
```

Now we have our doom binary compiled, we upload the binary and `doom1.wad` to `tmp` using our file-write primitive - we then run it in our stage 2 using `system()`. And we have hijacked the camera stream with DOOM!



(UN)SAFE AND SOUND

There are plenty of ways to get RCE on a device - WiFi, Bluetooth, SMS, etc. One method you don't hear about often (pun intended) is sound. In this article, we'll get RCE on a security camera using sound and leverage it to gain a root shell.

Sonic Pairing

I've already talked about the device featuring in the article - the camera I got to play DOOM on its stream. Who knew you could have so much fun with a £15 camera?

When the camera is reset, it enters a mode which constantly waits for WiFi credentials that can be delivered using a few methods - one of which is called 'Sonic Pairing'.

This feature uses frequency modulation magic to encode the given WiFi password, SSID, and bind key, into a sound. The user's phone plays this sound near the camera, and using the built-in microphone, it detects and decodes it. The decoded credentials are then used to connect to the network.

One of the hints that made me take a closer look at this surface (aside from being sound-based) is that, at the time, it was listed as 'beta' on the Yi IoT app.

Generating Custom Sounds

The app obviously limits the format of data modulated into the sound - it isn't possible to send bytes that are not valid characters. We need to generate sounds with fully controlled contents to exploit anything memory-corruption related.

Jadx was used to locate functions related to sound generation by focusing on relevant strings and similar indicators. I eventually came across a function that calls the *com.ants360.yicamera.util.PcmUtil.genPcmData* function.

This comes from a native library called *libpcmjni.so*. Luckily, frida (an awesome tool with many use cases) can be used to hook native functions. By hooking the function, we can replace the legitimate data that should get modulated onto the sound with our own.

As long as the data is no longer than 128 bytes, we can put (almost) whatever bytes we want into generated sounds with the following native hook:

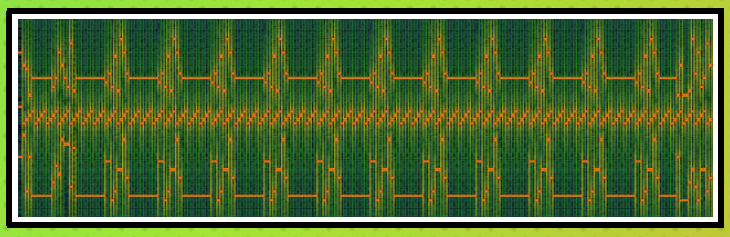
```
const ghidraImageBase = 0x100000;
const moduleName = "libpcmjni.so";
const moduleBaseAddress = Module.findBaseAddress(
    moduleName);
const functionRealAddress = moduleBaseAddress.add(0
    x103c4c - ghidraImageBase);
Interceptor.attach(functionRealAddress, {
    onEnter: function (args) {
        args[0].writeByteArray([
            0x62, 0xa, 0x41, 0x41,
            ...
            0x41, 0xa, 0x70, 0x0
        ]);
        console.log(hexdump(args[0]));
    },
    onLeave: function(args) {
        console.log("done");
    }
});
```

Bug 1: Stack Overflow

The sound wave parsing functionality is handled by the *sw_thread* in the main *anyka_ipc* binary, this always listens for sounds that resemble the expected format. Once received and decoded, it is split up into the *ssid*, *pwd* and *bind_key* components using \n as a delimiter.

At a high level, the string that can be provided by sound can be up to 128 bytes. In the function that handles these extracted credentials, there is a call to *sprintf(buffer, "ssid=%s", ssid)* where *buffer* is on the stack and has a capacity of 100 bytes (near the end of the stack frame) - giving us a small stack overflow.

As the *anyka_ipc* binary has ASLR on shared libraries and we have to worry about null terminators, this bug isn't enough to exploit by itself. As a PoC, I used the DOOM OOB-read to leak a pointer (which is probably cheating as the hotspot is required). However, with this I was able to jump back into the stack buffer, and execute a simple payload that turns the light on, sleeps, then crashes - here is a waveform of my exploit (never thought I'd say that).



Bug 2: Global Overflow

The second discovered bug is another simple one, this time a *strcpy* overflow on the processing of the decoded '*bind_key*'.

Initially, I didn't think this would be useful, but after spending some time investigating how the globals we can overflow were being used during the pairing process - I hit the jackpot.

As long as the first two characters of the *bind_key* are '*CN*' (which also makes the camera speak Chinese), we go down a code path that uses a '*did*' value we can overwrite with the overflow. This is used in a constructed command that gets executed via *system()* during the pairing process.

This means we can turn this global overflow into a reliable command injection, without requiring any other bugs!

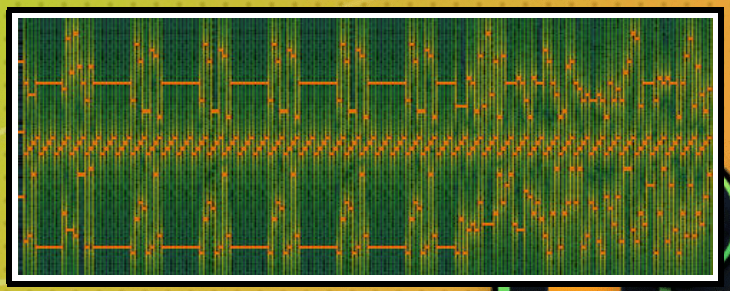
Exploiting for Root Shell

So how can we exploit this to get a root shell on the camera? The easiest way is to construct a sound that executes the *telnetd* command, and then we can connect with the unchangeable default credentials once it has connected to the network we control.

All we need is a WiFi hotspot that we control (it doesn't need to be connected to the Internet), and the *wav* file containing the exploit. We can then do the following steps:

- Get near the camera
- Play the sound at the camera
- Wait until the camera connects to your hotspot
- Get the IP of the camera, login to the telnet with *root* and no password
- Profit

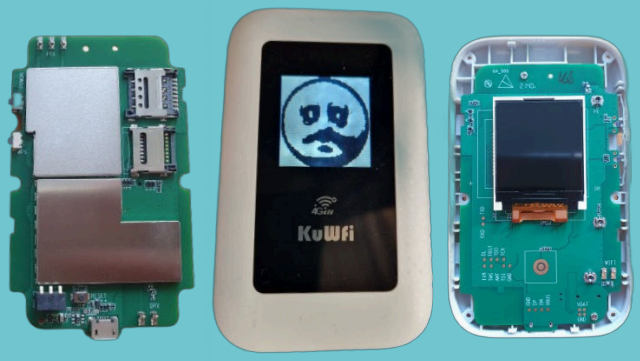
Obviously this only works when the camera isn't connected to the cloud, so not very useful - but pretty cool nonetheless. Here is the waveform of the second exploit!



Renaming All the Way to Root

While browsing Aliexpress, I stumbled upon a travel router featuring a screen - this immediately caught my interest, and I knew I had to buy one. In this article, we will go from LAN to code execution as root by chaining two of the several bugs I found.

Target
This travel router is branded as KuWfi C920, but it is actually a generic ZTE 4G travel router based on the ZX297520V3 chip (the specific model is MF910W).



There are several variants that use this chip, characterized by distinct visual features - some variants resemble traditional routers, while others incorporate screens, LED indicators, or expandable storage through SD card slots.

Getting Code
Before looking at any code, we first need to obtain the code itself. By removing the metal covers visible in the preceding images, we can access the WSON-8 Paragon memory chip. Once removed and mounted to a breakout board, we can dump its contents with a T48 programmer and then reinstall it.

Here is a visualization of the chip contents' entropy, created using *binvis.io*:

With the contents dumped, we remove the Error-Correcting Code (ECC) and pass it through Unblob, which extracts several file systems - including a few false positives. Our focus is on the *rootfs* ubifs filesystem, which contains the *GoAhead* web server binary that we want to analyze. With this in hand, we can now feed it into Ghidra and begin identifying bugs!

Pre-auth Arbitrary File Rename
The web server features a */goform* handler, which includes a *goform_set_cmd_process* method with numerous sub-methods. A whitelist enables some handlers, such as LOGIN, to function without authentication.
This router also features an SD card, which enables HTTPSHARE - essentially a small-capacity NAS on the router. Interestingly, this functionality allows files to be shared with unauthenticated users, making many HTTPSHARE *goform* methods accessible without prior authentication.
One of the methods is *HTTPSHARE_FILE_RENAME*, and it takes two parameters: *OLD_NAME_SD_CARD* and *NEW_NAME_SD_CARD*. Both of these are file paths.
The vulnerability lies in the lack of directory traversal checks, allowing an attacker to rename any file on the filesystem (excluding read-only files). Another bug that makes this more powerful, is that all the handlers are accessible even if the HTTPSHARE functionality is not enabled!
Here is an example request:

```
http://192.168.2.1/goform/goform_set_cmd_process
goformId=HTTPSHARE_FILE_RENAME&OLD_NAME_SD_CARD=/mmc
    2/./../../../tmp/test&NEW_NAME_SD_CARD=/mmc
    2/./../../../tmp/renamed
```

This means we can rename anything in */tmp*, */etc_rw* and */var*.

Authentication Bypass
So, can we use this to get the admin password? Yes we can!
There is another handler that allows fetching of the WiFi credentials encoded into a QR code, which is accessible pre-auth:

```
http://192.168.2.1/img/qrcode_ssid_wifikey.png
```

While directory traversal attacks are mitigated by GoAhead's URL handler, which strips out malicious characters, we can do something else.
The URL handler checks if the request is for the QR code image by using the *strstr* function, searching for the string *img/qrcode_ssid_wifikey.png*. If this check passes, it constructs a filename based on the output of the *strstr* call. The constructed filename typically points to */etc_rw/wifi/qrcode_ssid_wifikey.png*, but any characters after *.png* in the URL will be appended to the fetched filename.
The config file containing the admin password of the router is stored in */etc_rw/nv/main* as *cfg*. By crafting a URL that tricks the handler into constructing the desired filename, we can fetch the *cfg* file using the following steps:

1. Rename */etc_rw/wifi* to */etc_rw/wifi_backup*
2. Rename */etc_rw/nv/main* to */etc_rw/nv/qrcode_ssid_wifikey.png*
3. Rename */etc_rw/nv* to */etc_rw/nv/wifi*
4. Make a request to:

   ```
   http://192.168.2.1/img/qrcode_ssid_wifikey.png/cfg
   ```

5. Undo renames we did in steps 1-3 to cleanup
6. Extract the *admin_Password* item from the plaintext config we just retrieved, and login!

Other Pre-auth Bugs
Here are some other pre-auth issues I found that may be useful for bypassing the admin password some other way:

- File write (directory traversal)
- File delete (directory traversal)
- Directory create (directory traversal)
- Arbitrary config entry clear

Post-auth Bugs
Now that we have the admin password and have successfully logged in, we can get past the whitelist that limited our *goform* options before. With this increased attack surface, lets look for some bugs we could use for code execution:

- **Stack overflow**: *CHANGE_MAC*
- **11x Command injection (*Seriously?!?!*)**

There are probably more, but it feels a bit fruitless after the 11th command injection, so I stopped there!
Here is a request we can use to hit the *PINT_DIAGNOSTICS_START* command injection (up to 32 characters):

```
http://192.168.2.1/goform/goform_set_cmd_process?
goformId=PINT_DIAGNOSTICS_START&ping_diag_addr=127.0.0.1&
    ping_repetition_count=1&ping_time_out=1&ping_data_size
    =1&ping_diag_interface=eth0% ;
    *INJECTED COMMAND HERE* ;
```

Flappy Bird
So now we have gone from pre-auth LAN access, to remote code execution as root user - what could we do with this?
Obviously if we have a device with a screen we need to write a game for it - unfortunately we don't have many buttons to work with here. For this reason I chose to implement Flappy Bird as you only need a single button!
To implement this, I reversed how the screen framebuffer was being written (*/dev/fb0*), and how button presses worked. I then threw together a very basic flappy bird implementation, which turned out great!

No Chip-Off? No Problem

Sometimes hardware/dumping chips just isn't an option (until I can get my hands on some better kit). This article will go over my process from going to pre-auth LAN access, to remote code execution as root, on some ZTE travel routers - all without touching hardware.

Affected Devices
In terms of impacted devices, all of the bugs mentioned work on the following based on the **MT6735M** chipset (but there could be more):

- **MF904** (MF904Q-6.0-1-6735WM_LCD_20240123)
- **MF931** (MF931Q-1.0-1-6735WM-LED-20240301)

Getting Files via Directory Traversals
Without hardware, a chip dump isn't feasible. There isn't a nice exposed ADB via the USB connector either, so the only way to get started was a good ol' packet capture with Wireshark.

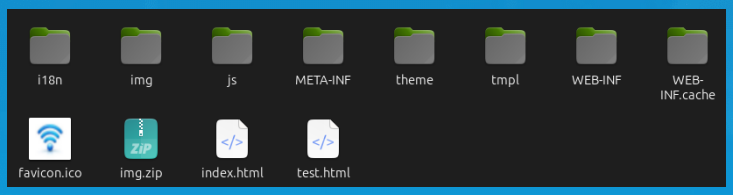
I eventually found a *Driver Download* page that does a POST request to */getfileForm*, surely this can't download any file I want?

```
POST /getfileForm?filename=../../../../../system/build.prop
```

The above request succeeds and returns the */system/build.prop* file with absolutely no issues. This functionality is post-auth so it probably wouldn't be useful for exploitation, but it is great for enabling further research.

How do I know what files there are to download? Well, there exists another post-auth method called *HTTPSHARE_ENTERFOLD* which allows listing of directories on the device. This handler has the exact same directory traversal issue, which means we can run *ls* in the root directory, and recursively download files, or recurse into subdirectories to get all of the files we can.

With the files fetched, the code for the web server was eventually discovered in */etc/TLR.db*.



Device Characteristics
It turns out these devices are based on the **MT6735M** chipset, and they look to be running Android 6.0. There are several apps that can be loaded into Jadx and reverse engineered.

All of the endpoints accessible via the HTTP server are located in */WEB-INF/web.xml*, and this also specified the associated classes in the *com.lr.web* application (another app on the device responsible for handling HTTP requests). This makes reverse engineering the handlers much simpler.

Post Auth Security Issues
After looking through the handlers, I found a couple of interesting issues:

- *Enable ADB over WiFi* : By making an authenticated request to */adbWifiDebugForm.do*, it will start a remote ADB process that you can connect to with *adb connect *ip* 5555*, giving you a root shell.
- *Command Injection via goform_set_cmd_process* : By making an authenticated HTTP request to */goform_set_cmd_process*, and specifying the *ADD_IP_PORT_FILETER* command, a command injection can be achieved by injecting via the *comment* parameter which gets included in a shell command that is executed with no injection checks.
- *Directory Traversals* : There are several directory traversals on this device, allowing reading/writing of files (like we did earlier).

I imagine there is much more to be found!

Authentication
Now that we have some post-auth issues, these won't be much use without some sort of authentication bypass.

Authentication has been correctly applied for the most part (on the *goform_set_cmd_process*), and only the following commands are available pre-auth:

- *LOGIN*
- *ENTER_PIN*
- *ENTER_PUK*
- *ENABLE_PIN*
- *DISABLE_PIN*
- *SET_WEB_LANGUAGE*

Most of the other endpoints check that the *loginfo* attribute is *ok*, otherwise they will redirect to the login page.

Authentication Bypasses
Although they have clearly put some thought into authentication, they've completely destroyed it with two issues.

The first bypass is a trivial hardcoded credential issue (some might say a backdoor?). Long story short, you can use *hebangadmin* as both the username and password to login to the admin panel!

The second issue is a little bit less trivial. So they have added authentication for pretty much all of the methods, except a pretty important one: *goform_get_cmd_process*.

What exactly can you do with this endpoint? Well, you can just ask nicely for the admin password:

```
GET /goform/goform_get_cmd_process?multi_data=1
    &cmd=admin_Password
```

The device will respond with the admin password, which you can then use to login to the device.

Conclusions

- You don't need fancy hardware for every project
- Simple bug classes are still around (Rust won't fix these :))
- Don't put hard-coded credentials on your products
- Steer clear of Aliexpress travel routers

strcpy(d,s); *cb-=4; // Gameboy

Does an action/body camera really need a WiFi hotspot? Either way, in this article, I will detail how I turned a heap overflow into a 4-byte decrement, and how I used this primitive to start a Gameboy emulator task to play some Pokemon!

Device

I was looking around Aliexpress for some devices to mess with, and I ended up coming across an action/body camera with a WiFi hotspot - this immediately piqued my interest.
The device seems unbranded and doesn't have a name/ID, but it is sold by *WEOU Camera Official Store* as a 4k Mini Camera. When a device that does not really need a hotspot has a hotspot, you can guarantee some dodgy code is handling those requests.
After hooking up to the UART, I was presented with an *msh* shell, indicating that the device is using RT-Thread - an open-source real-time operating system - specifically version *4.0.1*. More digging revealed the use of an ARM chip.

Heap Overflow

After some searching, I collected some bugs and useful primitives. The heap overflow to be used in this article is a pretty trivial *strcpy()* of the *Range* HTTP header into a fixed buffer within a struct of size *0x144*.
When we trigger the bug, we overflow a *64* byte buffer within the struct, a pointer to another struct, then outside of the allocated memory.

Exploiting for Limited ROP-Chain

Rather than diving into a complicated remote heap groom, I decided to see what primitives we get from the overwritten pointer. It turns out that we can leverage this to decrement an address in memory by a maximum of *13* (any more than this, and the hotspot will lock up due to not freeing the connections, so we only get one shot).
So what can I decrement to get execution? Fortunately, when an HTTP endpoint handler function is registered, the pointer to the function is stored at a fixed address in memory. There is a writeable pointer that points to the */index.html* handler function prologue. The instruction immediately before this is in another functions epilogue, in which it calls:

```
ldmia sp!,{r4, r5, r6, r7, r8, r9, r11, pc}
```

Due to the way the stack frame is laid out, we can get control of the would-be contents of these registers, and execute a ROP-chain (as *ldmia* moves the stack pointer further into our controlled buffer).
By sending a 'groom' request with an invalid method, and our bytes after *\r\n*, it seems to handle this as a separate request, so as long as we don't send a null terminator, we get full control of the *256*-byte buffer the registers are popped from.
As we can force the request to use the same session slot (they are deterministic), and the stack is not being modified between the 'groom' and 'trigger' requests, we can do the following:

1. Send four requests that use the overflow to decrement the pointer to the */index.html* callback; these connections must not be closed.
2. Next, send the HTTP request with the invalid method, with the contents of the buffer (a ROP-chain), close this connection immediately to free it for the next request.
3. Now make a valid request to */index.html* to trigger the modified callback, pop the registers we control, and execute a ROP-chain.

More ROP-Chain

Now we have a ROP-chain of *256* bytes, and a bunch of bad characters to contend with. Can we get something less constrained? How about that big *1024*-byte buffer that the request is *recv*'d into?
To do this, we just have to do some stack pivoting, which I was able to just scrape together in the constrained ROP-chain we have - shout out to this stack locator gadget:

```
| 0xc05af011 | add r0,sp,#0x20 | blx r6 |
```

Shellcode

With our unconstrained ROP-chain working, the next target is shellcode. For this, I reversed how memory was being initialised, and came across a function that was changing memory attributes. This led me to code that maps shared objects into memory for execution, so I copied the attributes from this and applied them to some allocated memory (this was done in the ROP-chain).
But how could I get code onto the device? Well, I used *clang* to build a binary, and used some makefile and linker script magic to get it into a format that could be directly executed (as if it was a function). I then used a file-write primitive I had found earlier to remotely write a file on the SD card called '*/mnt/sdcard/test.aac*' - this path was already in the binary, which saved me from having to put my own path somewhere.
So all I had to do was *malloc* some memory, change the attributes to be executable, load the contents of */mnt/sdcard/test.aac* into the buffer, spin up a separate thread with the loaded binary as the function to be executed, fix up the state, and return.

Building the Gameboy Emulator

While searching for usable Gameboy emulators, I came across *https://github.com/zid/gameboy* (a Gameboy emulator written in C), which looked perfect for the job. It required some work to port things like the display and buttons to the action camera, but it turned out great.
I used *clang* again, and some more makefile and linker script magic to compile it as a shared object with specific alignments (which is basically what a *.app* is on RT-Thread) that could be executed as an app in the shellcode.

Running Pokemon Red

So to run Pokemon Red, the following steps were completed (starting from code running in the thread spawned by shellcode):

1. Open a socket and receive the built Gameboy emulator app (sent from Python script).
2. Next, receive the ROM to be played, in this case Pokemon Red (also sent from Python script).
3. Now, load the entry function for the Gameboy app using *dlopen* and *dlsym*.
4. Execute the entry function to start the emulator!

If you got this far, go and check out the code!
https://github.com/lr-m/Action-Cam-Hacking

KILLING CANARIES FOR KIRBY

Hacking an IoT Camera to Play NES Games

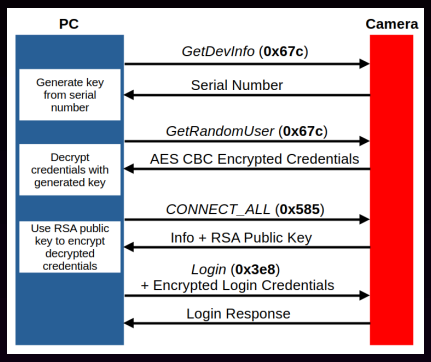
While browsing online, I found an intriguing camera that features a TFT screen for video calls using the iCSee app. This article will summarise discovered bugs and the strategy I used to hack it to play NES games remotely.

Logging In

By connecting the camera to a LAN, I could capture the comminications with the iCSee app. Most communications take place via TCP port 34567, and the protocol uses JSON heavily.

Once a connection is established, communications are encrypted, but this can be forced to plaintext by toggling a flag in the connection message.

From an unathenticated context only a subset of handlers are accessible, to unlock more, you must log in with 'random' credentials. To get these, you have to jump through hoops to decode/decrypt the credentials - these only work on the LAN as they are *Guest* credentials, but this expands the attack surface.



Port 34567 Bugs

The UART on the camera is only active during boot, the bootloader is password-protected - so a chip dump was necessary.

The camera is running Linux, and most of the functionality is handled by the */usr/bin/App* process, including the 34567 message handlers. In terms of mitigations, there is a stack canary, ASLR on the shared libraries and stack, but no NX.

After some auditing, the following useful bugs popped out:

- ***File Upload*** : Can remotely upload files to the SD card (a few hardcoded locations)
- ***Execution of SD Card File*** : By sending a message, if *iperf/iperf* is on the SD card, it will be executed
- ***Stack Overflows*** : Ten string-based stack overflows, only a single one was useful in this case
- ***Weak Canary*** : Instead of using the random canary, they use the address of the canary as the canary, which is fixed...

Canary Bypass

As mentioned above, they use a fixed canary (the address of the canary). Luckily for them, most of the stack overflows are string-based and there is a null terminator in the address (*0x0065b230*).

However, one of the discovered stack overflows has a great quirk that lets us fix the canary. The handler which contains the overflow is used to connect the camera to a specified FTP server and upload a log file. There is an overflow in the handling of the 'Name' parameter.

The code responsible for parsing the name has interesting logic, where if it encounters a '.', it will replace it with a count of the number of characters after the dot before (or start of the string). This means if we put '..' in the name, the second dot gets replaced with '0x00' - this lets us fix the canary and get control of the program counter!

Exploit Strategy

Now that we can work around the canary, we should be able to execute shellcode easily as there is no NX. The camera has an HTTP server (not really used for anything), but if you send it a request, a username and password are extracted from the request into executable global buffers at known locations. There is also an allocated buffer of *0x20000* bytes that the entire request ends up in, a pointer to this buffer is stored at a known fixed location. As the heap is also executable, we can place our main payload here.

1. First, send the HTTP request to stain the username/password buffers with stage 1
   - Username/password buffers contain a small ARM thumb payload (stage 1) to locate the *0x20000* buffer the request ends up in
   - Stage 2 is also sent in this request, offset into *0x20000* buffer is known
2. Then, trigger the stack overflow that fixes the canary to get control of pc
3. Camera will execute the thumb stage 1 to locate the stage 2, then will jump to it
4. Stage 2 then spawns a thread with code we want to execute (lets call it stage 3), then fixes up execution to return camera to a stable state

Porting smolnes

I found smolnes on github, which is a small NES emulator, and decided this would work well. Most of the effort for the porting was replacing SDL with code for interacting with the camera display (finding framebuffer in memory, etc). The interface between the smolnes and stage 3 was done using a couple of FIFO pipes, these are used for controls and display IPC.

It worked well, but it was slow, various compiler optimsations and rewriting some of the slow code helped speed it up.

Result

In the end, I used the File Upload primitive to send the *smolnes* binary and the *.nes* file, uploaded a script to run smolnes and executed it (using SD card file execution bug). Then the memory corruption exploit sets up the stage 3 to interact with the *smolnes* binary and draw the output to the screen.

Control inputs are received from stage 3 (which are sent to the camera via a Python script), and forwarded into the emulator to make the games playable.

Overall, it worked well, but there are probably easier ways to get your NES fix!

Check out the code!
github.com/lr-m/iCSeeU

DEAD AND KICKING:
Hacking ARM Mali Utgard Drivers

The Vulnerability

I started looking at this driver as a means to root the T11 translator, I pulled the device firmware down using *mtkclient* and looked for drivers to audit. I discovered it was using Mali Utgards r6p2 driver, and decided to research it as it hadn't been looked at much and source code is easy to acquire, plus I can access it from the *adb shell* environment.

The vulnerability I found is a use-after-free caused by a logic bug in the refcount handling for a *mali_mem_allocation* object. This object is created when a *MALI_IOC_MEM_ALLOC* ioctl is recieved, and maintains an internal refcount to keep track of, well, references. When *mmap* is called on the memory, the refcount is incremented, and vice versa when *munmap* is called.

When *MALI_IOC_MEM_FREE* is recieved, it will find the associated *mali_mem_allocation* object and call *mali_allocation_unref* on it. This will check that the *refcount - 1* is 0 (so no references), and free the underlying objects/memory if so.

```
_mali_osk_errcode_t _mali_ukk_mem_free(
    _mali_uk_free_mem_s *args)
{
  ...
  /* find mali allocation structure by
       vaddress*/
  mali_vma_node = mali_vma_offset_search
       (&session->allocation_mgr, vaddr,
        0);
  ...
  mali_alloc = container_of(
       mali_vma_node, struct
       mali_mem_allocation,
       mali_vma_node);

  if (mali_alloc)
    /* check ref_count */
    args->free_pages_nr =
         mali_allocation_unref(&
         mali_alloc);

  return _MALI_OSK_ERR_OK;
}

u32 mali_allocation_unref(struct
    mali_mem_allocation **alloc)
{
  u32 free_pages_nr = 0;
  mali_mem_allocation *mali_alloc = *
       alloc;
  *alloc = NULL;
  if (0 == _mali_osk_atomic_dec_return(&
       mali_alloc->mem_alloc_refcount))
       {
    free_pages_nr =
         _mali_free_allocation_mem(
         mali_alloc);
  }
  return free_pages_nr;
}

u32 _mali_osk_atomic_dec_return(
    _mali_osk_atomic_t *atom)
{
  return atomic_dec_return((atomic_t *)&
       atom->u.val);
}
```

The bug is that when they check that the *refcount - 1* is 0, they actually decrement the refcount with *atomic_dec_return*? So we can arbitrarily decrement this refcount, letting us free *mali_mem_allocation* objects early while there are still references to them.

Minnka: T11 Translator

The T11 translator is an Android-based device built around the MT6580 running Android version 7.0 and Linux Kernel version 3.18.15. The adb shell is locked down with a code, but this can be recovered using mtkclient.

I wrote this exploit first, and it takes advantage of a fixed-offset decrement primitive. You can do the following:

1. Use *mmap* calls to map the GPU memory a bunch of times, causing the refcount in the associated *mali_mem_alloc* to be incremented
2. Use the bug to cause the *mali_mem_allocation* to be freed early by decrementing the refcount over and over again
3. Get something allocated in the now-freed *kmalloc-128* memory
4. Call *munmap* on the mappings we created earlier to decrement the value at offset *0x4c* which would be the refcount

I got lucky with the *ion_buffer* object (accessible via */dev/ion*) - it has an *sg_table* pointer at offset *0x4c*, which controls physical memory mappings. Since this *sg_table* lives in *kmalloc-64*, we can spray a fake *sg_table* before the legitimate one, then use the decrement to redirect the pointer to our fake object.

Another great feature of this device is that the kernel can access userspace memory, allowing us to define fake objects in userland that the kernel can use directly.

After reversing to get necessary values, we construct a fake *sg_table* that enables mapping arbitrary physical addresses (incl. kernel memory). We spray fake *sg_table* objects via *sendmsg* calls, then create legitimate *ion* allocations whose *sg_table* structures partially occupy *kmalloc-64* regions, so the sprayed *sendmsg* memory is pinned in place. By selectively freeing ion buffers at intervals, we create gaps in *kmalloc-64* so that the legitimate *sg_table* will be allocated after a fake *sg_table* (while the *ion_buffer* lands in the freed *mali_mem_allocation* memory).

With the victim *ion_buffer* and fake *sg_table*'s positioned, we exploit the vulnerability to decrement the pointer, use *mmap* to map kernel memory containing a function address (I used an unused driver's *read* handler), patch it to point to `commit_creds(prepare_kernel_cred(NULL))` in userland, and pop a root shell!

```
aeon6580_we_n:/data/local/tmp # whoami
root
```

Frels: Soyes XS11

The Soyes XS11 is another Android-based phone built around the MT6580, this one runs Android 6.0 and Linux Kernel version 3.18.19. It is comically small, featuring a 2.5 inch display.

I quickly discovered that the technique used on the translator does not work on this device. This Mali Utgard driver version sets important *ion_buffer* values to null when exploiting the decrement primitive, causing *mmap* to fail. The kernel also can't access userspace memory, making faking structures a bit less convenient. A new approach is needed.

The alternative is triggering a double-free by setting the UAF object's refcount to 1, then freeing again via *munmap*. However, this corrupts the freelist, and since *kmalloc-128* is frequently used, the kernel crashes quickly.

The workaround: allocate controlled data between the frees and hold it for a while. The */proc/gsl_config* driver's *write* call copies arbitrary-sized user data into a *kmalloc*'d region. Setting certain values keeps it allocated for *20ms* which will work. When freed, we spray *mali_mem_alloc* objects to reoccupy the space, avoiding freelist corruption as long as we don't free them.

A user-controlled *mali_mem_alloc* free gives us a write primitive: *mali_mem_allocation_struct_destory* calls *list_del* with our controlled list pointer, which calls *__list_del*. With both pointers writeable, we can replace a function address with a controlled pointer (kernel code appears writeable).

```
static inline void __list_del(struct
    list_head * prev, struct list_head
    * next)
{
  next->prev = prev;
  WRITE_ONCE(prev->next, next);
}
```

Now we can patch a kernel function address with a controlled pointer, but unlike the translator, we cannot access userspace memory. Instead, I used existing kernel code to construct a JOP-chain that calls `commit_creds(prepare_kernel_cred(NULL))`. This works because the patched function pointer takes user-controlled arguments, letting me get the dispatcher table address into registers and use gadgets to load from it.

Finally, we can patch the handler function address to the JOP-chain, call it from userland, and pop another root shell!

```
root@f202_f22_p10:/ # whoami
root
```